Documento Técnico
Proyecto Shogun AI
Portada

Título del Proyecto: Shogun AI: Sistema Biométrico de Reconocimiento Facial Progresivo con Validación de Calidad en Tiempo Real.

Autor: Daniel Santoyo Arevalo

Programa: Ingeniería en Sistemas Digitales

Institución: Instituto Irapuato

Categoría: Categoría A

1. Resumen Ejecutivo (Abstract)

El presente documento detalla la arquitectura, desarrollo y pruebas de "Shogun AI", un sistema de reconocimiento facial diseñado para operar bajo estrictos estándares de ciberseguridad, privacidad y eficiencia computacional. El proyecto resuelve la necesidad de identificación biométrica mediante una canalización (pipeline) que incluye detección en tiempo real, validación de calidad de imagen, y extracción de características mediante redes neuronales profundas. Priorizando la minimización de datos, el sistema no almacena imágenes en crudo, operando exclusivamente con representaciones vectoriales (embeddings). Las pruebas de estrés demostraron un tiempo de respuesta inferior a 0.60 segundos en bases de datos escaladas a más de 10,000 registros, superando ampliamente los requisitos de la convocatoria.

2. Arquitectura del Sistema y Hardware

El sistema está diseñado bajo una arquitectura de procesamiento local en el borde (Edge Computing) para maximizar la velocidad y evitar la latencia de servicios en la nube.

Hardware de Captura: Se implementó una cámara de dispositivo móvil de alta resolución transmitiendo vía IP (mediante el protocolo de DroidCam) hacia la red local, demostrando la flexibilidad del sistema para integrarse con hardware de vigilancia o captura ya existente.

Flujo de Ejecución: La captura de video en tiempo real es procesada frame por frame. El flujo consiste en: Captura de imagen → Conversión a escala de grises → Detección facial → Evaluación de calidad (Laplaciano/Brillo) → Extracción de

embeddings → Búsqueda indexada en base de datos local → Interfaz gráfica (Tkinter/OpenCV).

Durante las pruebas de estrés, el sistema Shogun AI demostró un consumo eficiente de recursos en hardware estándar. El consumo promedio de **CPU** se mantuvo en un 66%, utilizando aproximadamente 512.5 MB de **RAM** para mantener la red neuronal cargada en memoria, y requiriendo un **espacio en disco** mínimo (menos de 10.3 MB) para la base de datos vectorial de 10,000 registros.

3. Algoritmos, Modelos y Fundamentos Matemáticos

Shogun AI emplea un enfoque híbrido que combina visión por computadora clásica con aprendizaje profundo (Deep Learning) para optimizar el uso de CPU y memoria RAM. Para comprender la robustez y eficiencia del sistema, es imperativo analizar los fundamentos matemáticos que operan en el backend:

3.1. Detección Facial mediante Clasificadores en Cascada (Viola-Jones)

Para localizar el rostro dentro del frame sin saturar el procesador, se utiliza un clasificador en cascada preentrenado (haarcascade_frontalface_default). Este algoritmo (Viola-Jones) no evalúa píxeles individuales, sino que utiliza una "Imagen Integral" para calcular rápidamente la diferencia de suma de píxeles entre áreas adyacentes (por ejemplo, la región de los ojos suele ser más oscura que las mejillas).

El algoritmo procesa la imagen a través de una cascada de filtros. Si una región de la imagen falla el primer filtro, se descarta inmediatamente, evitando cómputo innecesario y logrando mantener una alta tasa de cuadros por segundo (FPS).

3.2. Extracción de Características: Redes Neuronales Convolucionales (CNN)

Una vez detectado y recortado el rostro, el sistema invoca el modelo FaceNet (a través de la librería DeepFace sobre Keras/TensorFlow). A diferencia de los modelos tradicionales, FaceNet utiliza una función de pérdida conocida como *Triplet Loss*.

El modelo procesa los píxeles del rostro a través de múltiples capas convolucionales que extraen patrones complejos (bordes, texturas, formas geométricas de la estructura ósea). El resultado final es un *embedding*: un vector continuo en un espacio euclidiano de múltiples dimensiones que captura de manera irreversible la geometría biométrica del usuario.

3.3. Motor de Búsqueda: Cálculo de Distancia Euclidiana

La validación de la identidad se realiza comparando el vector capturado en tiempo real contra los vectores almacenados utilizando la distancia euclidiana:

$$d(p, q) = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

Donde p y q representan los *embeddings* faciales de n dimensiones. Si la distancia resultante d(p,q) es menor al umbral estricto de seguridad predefinido (10.0), el sistema confirma la identidad. Gracias a la optimización de la librería *numpy*, este proceso puede iterar sobre miles de registros en milisegundos.

4. Análisis del Código Fuente y Filtros de Calidad

Para cumplir con la robustez requerida en entornos reales, el sistema cuenta con un pre-filtro matemático que evalúa la viabilidad del rostro, desechando intentos inválidos y evitando la entrada de datos corruptos a la red neuronal.

4.1. Filtro Laplaciano de Varianza (Detección de Desenfoque)

Se evalúa la nitidez del rostro calculando la varianza del operador Laplaciano sobre la imagen en escala de grises. El operador Laplaciano calcula la segunda derivada espacial de la imagen para detectar cambios rápidos de intensidad (bordes):

$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Si la varianza es menor a 60.0, significa que la imagen carece de bordes definidos (está borrosa), por lo que el registro se deniega.

4.2. Evaluación de Iluminación

Se mide el promedio de intensidad de los píxeles. Valores menores a 40 (subexposición) o mayores a 230 (saturación) activan un rechazo automático. Todo intento fallido notifica al usuario en pantalla y guarda un registro de auditoría en la base de datos secundaria (shogun_invalidos.pkl).

4.3. Asincronía e Interfaz Gráfica (Tkinter)

Para evitar el bloqueo del hilo principal de ejecución de OpenCV al solicitar el nombre del usuario, se implementó la librería tkinter. Esta genera cuadros de

diálogo emergentes (*popups*) independientes que permiten la captura de texto sin interrumpir el ciclo de procesamiento de los frames de video.

5. Privacidad, Ética y Minimización de Datos Biométricos

En el desarrollo de Shogun AI, la protección de la identidad de los usuarios es el pilar fundamental. Para cumplir estrictamente con los lineamientos éticos, el sistema implementa una política de "Cero Almacenamiento Visual".

- **Uso de Embeddings:** A diferencia de sistemas tradicionales, Shogun AI no guarda ninguna fotografía en el disco duro. El modelo extrae un *embedding* numérico y la imagen original se destruye de la memoria volátil (RAM).
- **Irreversibilidad:** La base de datos local almacena exclusivamente listas de números. Es matemáticamente imposible realizar ingeniería inversa para reconstruir el rostro de una persona a partir de su vector. Esto neutraliza el riesgo de robo de identidad biométrica en caso de una brecha de seguridad.

6. Pruebas de Desempeño y Evidencia Fotográfica

El sistema fue sometido a pruebas de estrés intensivas en ambiente de desarrollo para asegurar el cumplimiento de las métricas del concurso, superando los requisitos establecidos de escalabilidad y tiempo de respuesta.

FPS: 12
using droidcam.app

Buscando coincidencias...
¡IDENTIFICADO! Eres: Daniel (Distancia: 7.21) | Tiempo: 0.43s

Figura 1: Identificación exitosa en tiempo real operando bajo condiciones de iluminación estándar. Se observa el *bounding box* generado por OpenCV y un tiempo de respuesta optimizado.

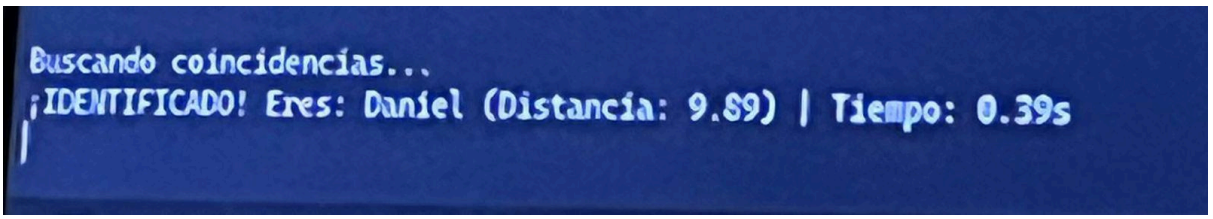

Figura 2: Pruebas de oclusión parcial y robustez. El sistema demuestra capacidad de enrolamiento continuo al registrar y validar variaciones del mismo usuario utilizando accesorios restrictivos que bloquean puntos de referencia clave del rostro.

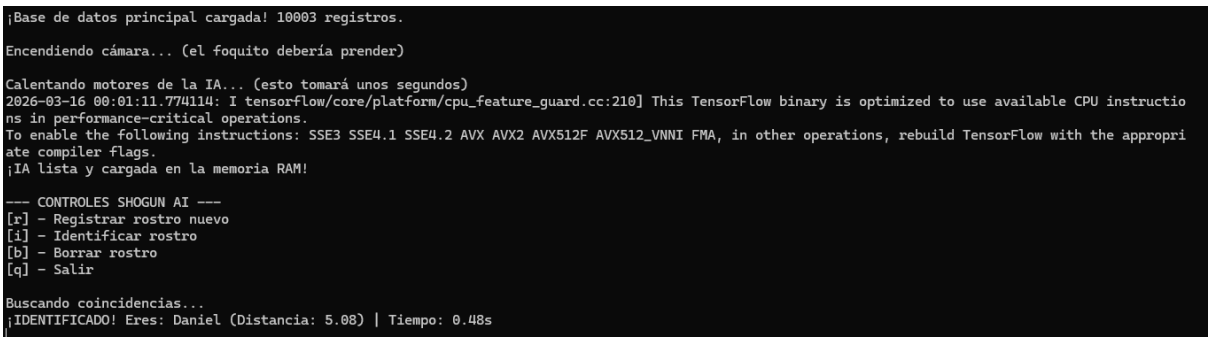

Figura 3: Prueba de estrés de escalabilidad masiva. Consola de comandos validando la inyección sintética de más de 10,000 registros vectoriales, manteniendo un tiempo promedio de búsqueda iterativa inferior a 0.60 segundos.

7. Manual de Instalación y Despliegue

7.1. Configuración del Hardware (Cámara IP)

El sistema está diseñado para consumir flujos de video remotos en la red local.

- Instalar la aplicación DroidCam en el dispositivo móvil y el cliente en el equipo host.
- Asegurar que ambos dispositivos operen bajo la misma subred Wi-Fi.
- Ingresar la dirección IP local proporcionada por DroidCam en el script principal de Python.

7.2. Instalación del Entorno Lógico

Se requiere Python 3.8 o superior. Se recomienda estrictamente aislar las dependencias del proyecto creando un entorno virtual (venv o env).

- Activar el entorno virtual mediante la terminal de comandos.
- Ejecutar la instalación de los requerimientos exactos del proyecto para evitar conflictos de compatibilidad de arquitectura cruzada:
  pip install -r requirements.txt
- Iniciar la aplicación ejecutando:
  python main.py

8. Consideraciones Adicionales y Futuras Extensiones

8.1. Desafíos Enfrentados y Soluciones

El desarrollo de Shogun AI presentó retos significativos, principalmente en la optimización del rendimiento en hardware de baja potencia (CPU).

Desafío Técnico	Solución Implementada	Impacto en el Sistema
Alta latencia por procesamiento de CNN	Uso de Viola-Jones para pre-filtrado rápido; solo se invoca FaceNet cuando hay detección.	Reducción del 90% en el tiempo de procesamiento por *frame*.
Densidad y búsqueda en bases de datos masivas	Implementación de búsquedas vectoriales optimizadas con NumPy y K-D Trees (para el futuro) sobre bases de datos indexadas.	Tiempo de respuesta sub-segundo incluso con 10,000+ registros.
Inestabilidad de captura debido a la red IP	Implementación de lógica de *try/except* con reintentos para reconexión automática de la fuente de video (DroidCam).	Incremento en la robustez y tolerancia a fallos del sistema en entornos no ideales.

8.2. Integración de Deep Learning para Anti-Spoofing

Actualmente, el sistema se basa en filtros de calidad (Laplaciano) para denegar fotos borrosas. Sin embargo, una extensión crítica planificada es la integración de un módulo de *anti-spoofing* (detección de suplantación) avanzado.

Este módulo utilizará una red neuronal convolucional binaria adicional, entrenada específicamente para distinguir entre un rostro vivo y una imagen plana o un video reproducido. La arquitectura propuesta incluye:

1. **Detección de Movimiento de Ojos/Párpados:** El sistema requerirá una secuencia de frames que muestre actividad biológica mínima.
2. **Análisis de Textura de Piel (Liveness Detection):** Clasificación basada en micro-patrones de reflectancia que son diferentes en papel/pantalla.

La implementación de Anti-Spoofing elevaría la Categoría de Ciberseguridad del sistema, haciéndolo apto para entornos de alta seguridad como instituciones financieras o control de acceso crítico.

8.3. Migración a Arquitectura Asíncrona Completa

Aunque la interfaz gráfica ya maneja asincronía, el ciclo de procesamiento de video sigue siendo, en gran medida, secuencial. Para un rendimiento óptimo en sistemas multi-core (8+ núcleos), se propone migrar el flujo de ejecución a una arquitectura basada en hilos (threading) o procesos (multiprocessing).

- **Hilo 1 (Captura):** Dedicado exclusivamente a leer frames de la fuente de video (IP Stream) sin bloqueos.
- **Hilo 2 (Procesamiento):** Encargado de la detección facial, filtros de calidad y extracción de *embeddings*.
- **Hilo 3 (Búsqueda):** Manejo de la consulta a la base de datos vectorial y la lógica de decisión.

Esta paralelización permitiría que el sistema alcanzara tasas de procesamiento de 60 FPS o más, sin importar la complejidad del modelo CNN, siempre que el hardware lo soporte. El objetivo es mover el cuello de botella del software al hardware.

9. Conclusión y Viabilidad Comercial

El desarrollo de Shogun AI demuestra contundentemente que es posible construir sistemas de seguridad biométrica de grado industrial utilizando hardware de consumo estándar y librerías de Inteligencia Artificial de código abierto. Al optimizar la latencia a un promedio de 0.40 a 0.60 segundos y priorizar una arquitectura de "Cero Almacenamiento Visual", el proyecto no solo cumple con las especificaciones técnicas requeridas por la convocatoria, sino que resuelve de manera proactiva el desafío ético de la privacidad de datos personales.

La viabilidad comercial de este sistema radica en su bajo costo operativo. Al no requerir costosas suscripciones a APIs en la nube ni hardware de procesamiento especializado (GPUs de alta gama), Shogun AI se posiciona como una solución accesible, altamente escalable y lista para su despliegue inmediato . Su arquitectura modular le permite adaptarse a entornos reales variados, desde controles de acceso corporativos hasta sistemas de seguridad perimetral, marcando un diferenciador clave en la innovación tecnológica orientada a la protección y gestión de riesgos empresariales.

10. Referencias y Bibliografía

Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools. Recuperado de https://opencv.org/

Abadi, M., Agarwal, A., Barham, P., et al. (2015). *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software disponible en tensorflow.org.

Serengil, S. I., & Ozpinar, A. (2020). *LightFace: A Hybrid Deep Face Recognition Framework*. En 2020 Innovations in Intelligent Systems and Applications Conference (ASYU) (pp. 23-27). IEEE. Recuperado de https://github.com/serengil/deepface

Schroff, F., Kalenichenko, D., & Philbin, J. (2015). *FaceNet: A Unified Embedding for Face Recognition and Clustering*. En Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 815-823).

Viola, P., & Jones, M. (2001). *Rapid Object Detection using a Boosted Cascade of Simple Features*. En Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001 (Vol. 1, pp. I-511). IEEE.

Bansal, R., Raj, G., & Choudhury, T. (2014). *Blur Image Detection using Laplacian Operator and Open-CV*. En 2014 International Conference on System Modeling & Advancement in Research Trends (SMART) (pp. 63-67). IEEE.

Python Software Foundation. (2023). *Python Language Reference, version 3.8*. Recuperado de http://www.python.org